

SQL

(MySQL, PostgreSQL, Oracle, SQL server)

Instruccions MYSQL:

- `mysql -u root -p` -> entrar a mysql
- `show databases;` -> mostrar bases de dades
- `create database "nom";` -> crear bases de dades
- `drop database prova;` -> eliminar bases de dades
- `use "nom";` -> entrar a bases de dades
- `show tables;` -> mostrar tables
- `select database();` -> mostrar base de dades actual
- `create table "nom";` -> crear tabla.
- `create table "nom" ("nom columna de dades", nom columna de dades);`
- `create table "nom" ("nom columna de dades" "tipus de dades" "unsigned", nom columna de dades "tipus de dades");` (enters podem fer servir unsigned si no vol utilitzar números negatius) (varchar assignar la longitud máxima (0)).
- `drop table "nom";` -> eliminar table.
- `show warnings;` -> mostrar errores.
- `desc t1;` -> mostrar el contingut de la taula.
- `f7` -> muestra l'historial de instruccions introduïda.
- `system cls` -> netejar la terminal.
- `source + archivo` -> crear tables fetes en un arxiu.
- `create table IF NOT EXISTS "nom"` -> crear table i verificar si no exists.
- `insert into nomTable (dades separat amb comes);` -> guardar dades en un table.
- `default` -> per assignar-li un valor per defecte a una columna de la tabla.
- `select * from nomTable` -> seleccionar tots els valors de la tabla.
- `primary key` -> per indicar que la columna s'identifica per una clau.
- `null` -> els valors de una columna poden ser null (valor no obligatori).
- `not null` -> els valors de una columna no poden ser null (valor obligatori).
- `unique` -> no s'accepten valors repetit, però si null (és el mateix que primary key, només que aquest accepta valors nulls).
- `check` -> validar els valors de les columnes (la condició és en base al que vulguis comprovar).
- `auto_increment` -> s'utilitza només a la clau primari; incrementa el valor en 1 a cada id.
- `comment` -> s'utilitza per posar comentaris a les columnes.
- `constraint` -> restriccions per les taules amb la següent sintaxis: `CONSTRAINT "nom" Tipus(columna(es)).`
- `delete from (nomTaula) where (condicio)` -> per a eliminar una fila de la taula.
- `update nomtable set nomFila = "" where nomFila = ""` -> per a actualitzar una fila de la taula.
- `rename table nombre to "nuevo nombre"` -> Cambiar el nom a una taula.
- `alter table "nombre" add column ""nom column' 'tipus de dada(longitud)''` -> per a agregar un atribut a la taula.
- `alter table "nombre" drop column "nom column"` -> per a eliminar un atribut de la taula.
- `alter table "nombre" modify column "nom column" "definició columna"` -> per a modificar tota l'estructura (sense el nom) (tipus(longitud) restriccions) de un atribut de la taula.
- `alter table "nombre" change column "nom column" "nou nom column" "definició columna"` -> per a modificar el nom i tota l'estructura (tipus(longitud) restriccions) de un atribut en la taula.
- `alter table "nombre" add constraint "restricció"` -> per a agregar restriccions

a l'atributs de una taula.

- `alter table "nombre" drop constraint "restricció" ->` per a eliminar restriccions a l'atributs de una taula.

- Exportar base de dades -> `mysqldump -u root -p nom base de dades > exemple.sql`

MYSQL (vistas):

- Usar punt per separar reals.
- EN TIPUS ENTERS I REALS UTILITZA OPCIONALMENT (UNSIGNED): no usar números negatius.
- CHAR té longitud fixa
- VARCHAR és variable
- Si no introduïm valors a les columnes, per defecte serà null
- En SQL els operadors relacionals són els mateix, però per comparar utilitzar només un `=` i per diferent `<>` o `!=`.
- En SQL per verificar cadenes utilitzar cometes simples.
- En SQL els operadors lògics se anomenen igual que en Python. `and`, `or` i `not`.
- En les taules es guarden columnes.
- La clau primària no accepta valors null.
- En constraint si no posem el nom a la restricció, mysql el posarà.
- Not null no és una restricció de taules.
- Per a agregar el foreign key s'ha de tenir en compte dues condicions, que existeixi la taula i la clau primària.
- Fer servir constraint per les restriccions de les taules és el mateix que fer-ho en la columna.
- `constraint foreign key (nomColumna) references nomTaula(columna)`.
- Quan tenim la relació foreign key per a eliminar la taula que té la restricció, primer s'ha d'eliminar la taula que conté l'aneu de la relació.
- La restricció foreign key utilitza `on delete cascade` per eliminar les dades de la relació entre les taules.
- La restricció foreign key utilitza `on delete cascade set null` per eliminar la columna en la taula que permet la relació i deixa les files de la taula relacionat en null.
- La restricció foreign key utilitza `on delete restrict` per no permetre esborrar les files de la taula relacionat.
- S'utilitza `restrict` en `'on delete'` i `'on update'` que en `'on delete'` no permet esborrar, en `'on update'` no permet actualitzar.
- S'utilitza `cascade` en `'on delete'` i `'on update'` que en `'on delete'` permet eliminar una fila i el que depèn d'aquesta, en `'on update'` permet actualitzar una fila i actualitzar el que depèn d'aquesta.
- S'utilitza `set null` en `'on delete'` i `'on update'` que en `'on delete'` permet esborrar i el que depèn ho assigna a null, en `'on update'` permet actualitzar i el que depèn ho assigna a null.
- `alter table` funciona per a agregar, modificar i eliminar els atributs d'una taula. (separant per comes és possible agregar, modificar i eliminar més d'un atribut).
- En `alter table`, si utilitzem `first` per a agregar un atribut abans d'un altre.
- En `alter table`, si utilitzem `after` per a agregar un atribut després d'un altre.
- En `alter table`, per a modificar els tipus de dades no podem tenir menys longitud de la que teníem definida anteriorment.
- En `alter table`, si solo voy a modificar restricciones debo indicar el

tipo(longitud) que tenia definido el atributo.

- En alter table també es pot agregar i eliminar restriccions de taules amb constraints.

- Les restriccions des de alter table no es poden modificar.

Tipus de dades:

• Tipus Enter

TINYINT	1 byte	0 - 255 (-127 0 127)
SMALLINT	2 bytes	0 - 65,000
MEDIUMINT	3 bytes	0 - 16,000,000
INT	4 bytes	0 - 4,000,000,000
BIGINT	8 bytes	0 - 9,000,000,000,000,000,000

• Tipus Reals

FLOAT	4 bytes	0 - 4,000,000,000
DOUBLE	8 bytes	0 - 9,000,000,000,000,000,000

• Tipus Texts

CHAR	255 bytes
VARCHAR	255 bytes

TEXT ES DIVIDEIX:

TINYTEXT	255 bytes
TEXT	65535 bytes
MEDIUMTEXT	16777215 bytes
LONGTEXT	4*10 ⁹ bytes

• Tipus Data i Hora

DATE format (YYYY-MM-DD)
TIME format (HH:MM:SS)
DATETIME format (combinació de les dos anteriors)
YEAR format (2 o 4 dígit)

• Tipus boolean

Boolean

• Tipus desconegut

NULL

Restriccions:

• Restriccions

NULL | NOT NULL
PRIMARY KEY = KEY
UNIQUE = UNIQUE KEY
CHECK

• Opcions de columna

COMMENT
AUTO_INCREMENT

• Restriccions de taules

* CONSTRAINT *

PRIMARY KEY = KEY
UNIQUE = UNIQUE KEY
CHECK
FOREIGN KEY (on delete: restrict, cascade, set null. on update; cascade)

Exemple

```
drop table persona;
-- En el create podem indicar el valor per defecte
create table persona (
    id integer,
    nom VARCHAR(20) default Null,
    poblacio varchar(20) default 'No indicat',
    sou FLOAT default 1200.23
);
-- Si fem desc persona podem veure a default el valor assignat

-- Quan fem inserts si posem valors es respectaran
insert into persona values (1, 'Jordi', 'Barcelona', 1000);

insert into persona values (2, 'Moises', 'Barcelona', 1200.45);

insert into persona values (3, 'Oriol', null, null);
-- Si no indiquem valors aleshores es posarà el valor per defecte
insert into persona (id, nom) values (4, 'Alba');

-- Per mostrar la informació
select * from persona;
```

```
drop table coche;
drop table persona;

create table persona(
    dni char(9),
    nom varchar(30)
);

create table coche(
    matricula char(7),
    marca varchar(30),
    model varchar(30),
    dni_propietari char(9)
);
```

-- Afegir claus primàries

```
alter table persona
add constraint pk_persona primary key(dni);

alter table coche
```

```
add constraint pk_coche primary key(matricula);
```

```
-- -----
```

```
-- Afegir claus foranes
```

```
-- -----
```

```
alter table coche
```

```
add constraint fk_coche_propietari
```

```
foreign key (dni_propietari) references persona(dni);
```

```
--
```

```
insert into persona values('1111A','Moises');
```

```
insert into coche values('B1111A', 'Seat','Ibiza','1111A');
```

```
insert into coche values('B2222B', 'Mercedes','chulo',NULL);
```

```
delete from persona;
```

```
-----
```

```
-- MODEL 1
```

```
create table coche(
```

```
    matricula char(7) primary key,
```

```
    marca varchar(30),
```

```
    model varchar(30),
```

```
    dni_propietari char(9)
```

```
);
```

```
-- MODEL 2
```

```
create table coche(
```

```
    matricula char(7),
```

```
    marca varchar(30),
```

```
    model varchar(30),
```

```
    dni_propietari char(9),
```

```
    constraint pk_coche primary key (matricula)
```

```
);
```

```
-- MODEL 3
```

```
create table coche(
```

```
    matricula char(7),
```

```
    marca varchar(30),
```

```
    model varchar(30),
```

```
    dni_propietari char(9)
```

```
);
```

```
alter table coche
```

```
add constraint pk_coche primary key(matricula);
```

CONSULTAS SQL

set lc_time_names='Locale value' -> para cambiar el idioma en la sesion actual.

locale values -> <https://dev.mysql.com/doc/refman/8.0/en/locale-support.html>

show create table "nomTabla" -> para mostrar la estructura de la tabla.

* SELECTS *

- Se pueden seleccionar todos los valores. -> select * from nomTabla
- Se pueden seleccionar valores especificos. -> select nomColumna, nomColumna from nomTabla
- Se pueden realizar expresiones dentro de las consultas. -> select nomColumna 'operadores' valor o nomColumna from nomTabla
- Cuando seleccionamos mediante expresiones se crea una nueva columna para mostrar los datos requeridos, pero a esa columna se le puede asignar un nombre. -> select nomColumna 'operadores' valor o nomColumna as nomExpresion from nomTabla
- Cuando usamos los operadores aritmeticos el comportamiento es igual, excepto que en sql existe el operador div que permite realizar divisiones enteras. -> select nomColumna 'div' valor o nomColumna as nomExpresion from nomTabla
- La división por cero no retorna un error si no que retorna null.

* SELECT + WHERE *

- El where en el select permite filtra fila de datos en un cierto rango donde se cumple una determinada condición. -> select * from nomTabla where condicion
- En el where se utilizan los operadores de comparación excepto que existe el operador <> (distinto) que es igual que diferente !=. -> select * from nomTabla where nomColumna <> nomColumna o valor.
- En el where se utilizan los operadores de logicos igual que en programacion Python y Java. -> select * from nomTabla where nomColumna = valor 'and/||' 'or/||' 'xor' nomColumna = valor.
- En el where se utiliza BETWEEN y NOT BETWEEN que es lo mismo que utilizar los operadores '<' y '>', mejor dicho verifica que un valor este o no dentro de un rango.
- En el where se utilizan los filtros IN y NOT IN que verifica si un valor o dato esta dentro de un especificaciones establecido. -> select * from nomTabla where nomColumna 'in' 'not in' (valores).

TALLER SQL (CRUD)

SUSPECT_ID ???

NAME ???
SEX 'male'
AGE 50 and 60
HEIGHT ???
HAIR_COLOR 'brown' and 'black'
EYE_COLOR ???
FACIAL_HAIR ???
TATTOS ???
GLASSES 'yes'
SCARS 'yes'
FEET_SIZE 'large'

· select name from suspects where (sex='male' and age>=50 and age <=60) and (hair_color='brown' or hair_color='black') and glasses='yes' and scars='yes' and feet_size='large';

* name = 'Kasper Good'

SUSPECT_ID ???
NAME ???
SEX ???
AGE 36 and 40
HEIGHT ???
HAIR_COLOR 'red'
EYE_COLOR 'blue'
FACIAL_HAIR ???
TATTOS 'yes'
GLASSES
SCARS ???
FEET_SIZE 'small'

· select name from suspects where age>=35 and age<=40 and hair_color='red' and eye_color='blue' and tattoos='yes' and feet_size='small';

* name = 'Hilary Mayer'

Order by

Para obtener el orden ascendentes y descendente de los valores (tantos cadenas como numericos) de una columna -> select * from NomTabla order by nomCol asc o desc;

· Por defecto es ASC
· Cuando tengo varias columnas aplicara el orden a la primera si en esta hay empates (igualdad), pasara a aplicar el orden a la siguiente columna y asi sucesivamente hasta no encontrar empates (igualdad), en caso de todas las columnas tenga empates (igualdad) se quedara con el orden de la ultima.

* Funciones *

- Es posible jugar con las funciones, por ejemplo usar funciones dentro de otras funciones.

* Funciones numericas

ABS(num) -> valor absoluto del número.

CEIL(num) -> redondear hacia arriba el número ceil(6.23).

FLOOR(num) -> redondear hacia abajo el número floor(6.23).

MOD(num1, num2) -> obtener el residuo de la división.

RAND() -> obtener un número random entre 0 i 1.

ROUND(num[d]) -> redondear el número a partir de .5 es hacia arriba y .49 es hacia abajo.

• Por ejemplo si quiero obtener el redondeo de n (n indica el numero de decimales) decimales redondeara los decimales aplicando la misma norma anterior.

TRUNCATE(num, d) -> Elimina los decimales indicados.

* Funciones de caracteres

CHAR_LENGTH(str) -> número de caracteres de un String (cadena de texto).

• CHAR_LENGTH a diferencia de LENGTH no cuenta acentos y diéresis.

LENGTH(str) -> Longitud de caracteres en bytes (un valor numerico).

• LENGTH cuenta todos los caracteres incluidos acentos y diéresis.

CONCAT(str1, str2, ...) concatena caracteres a igual que java o otros lenguajes de programacion, su sintaxis es diferente pero hace lo mismo, en vez de concatenar con el signo + utiliza comas.

INSTR(str, caracter) busca la posición del carácter en la cadena y lo retorna.

LOWER(str) convierte una cadena en minusculas.

UPPER(str) convierte una cadena en mayusculas.

LTRIM(str) elimina los espacios del lateral izquierdo.

RTRIM(str) elimina los espacios del lateral derecho.

TRIM(str) elimina los espacios del lateral izquierdo y derecho.

LPAD(str1, num, str2) rellena la cadena str1 con str2 por el lateral izquierdo hasta que el str1 llegue a la longitud indicada en el num.

REPLACE(str, str1, str2) cambia todas las apariciones del str1 por str2.

REVERSE(string) invierte la cadena indicada.

SUBSTRING(str, ini, numCar) retorna una cadena desde la posición inicial y el numero de caracteres a tomar(si no se indica el numCar se mostrarán todos los

caracteres de la posición inicial hasta la final).

MID(str, ini, fi) es equivalente a SUBSTRING.

* Funciones de fechas *

NOW() para establecer la fecha actual automáticamente.

YEAR(data) para tomar el año de la fecha indicada.

MONTH(data) para tomar el mes de la fecha indicada.

DAY(data) para tomar el día de la fecha indicada.

TIME(data) para tomar la hora, minutos y segundos actuales.

HOURL(data) para tomar la hora de la fecha indicada.

MINUTE(data) para tomar los minutos de la fecha indicada.

SECOND(data) para tomar los segundos de la fecha indicada.

date_format(data, format) formatea la fecha indicada al formato que se indique, sigue convenciones de parámetros igual que String.format de Java.

El format puede ser:

%Y Any 4 digits - %y Any 2 digits

%M Nom del mes - %m Mes

%d Día del mes

%H Hora 24 horas - %h Hora 12 horas

%i Minuts

%s Segons

%W Nom día de la semana

%w Día de la semana (0 diumenge)

%p AM o PM

ADDDATE(data, dies) para sumarle un número determinado de días a la fecha indicada.

ADDDATE(data, interval) para sumarle un intervalo determinado puede ser horas, minutos, segundos, días, meses, años. por ejemplo: ADDDATE(data, interval 1 DAY, 1 MONTH o 1 YEAR o 1 HOUR, 1 MINUTE, 1 SECOND) el 1 puede variar según lo que necesites.

SUBDATE(data, dies) lo mismo que el ADDDATE(data, dies) pero retorna lo contrario.

SUBDATE(data, interval) lo mismo que el ADDDATE(data, interval) pero retorna lo contrario.

DATEDIFF(data1, data2) retorna la diferencia de días entre las dos fechas;

TIMESTAMPDIFF(int, d1, d2) retorna la diferencia en el intervalo indicado.

PARA EL EXAMEN

NUMERICS

- CEIL
- FLOOR
- ROUND
- TRUNCATE

STRING

- CONCAT
- CHAR_LENGTH
- UPPER / LOWER
- INSTR
- SUBSTRING

DATA

- NOW()
- YEAR, MONTH, ...
- ADDDATE
- DATEDIFF

Consultas SQL con mas de una tabla (unión de tablas)

SQL 2 o SQL 92

select * from 'nomTabla', 'nomTabla' -- Producto cartesiano -> Muestra todos los valores de la primera tabla combinada con la segunda tabla. Las combinaciones de id es una muestra de que la combinacion de las tablas es correcta.

ejemplo:

```
select *  
from 'nomTabla', 'nomTabla'  
where 'nomTabla'.'columna' == 'nomTabla'.'columna';
```

Las combinaciones funcionan tanto en el select, como en el from y el where.

Selección con tres tablas, mayormente se hacen cuando hay un tabla que es creada por la relacion N:N de dos tablas, para realizar la union es unicamente mediante la tabla de la relación.

Los alias tambien se utilizan en las tablas, mejor dicho en el from. No hay que poner la palabra reservada as.

SQL 3 o SQL 99

- Nuevos tipos de unión
- Mejorar la lectura

- Croos join: producto cartesiano.

- Natural join: unifica todos los atributos que sean iguales, pero es un problema cuando hay mas de un atributo con el mismo nombre en las diferentes

tablas.

- Join ON:

- Left outer join on: muestra todas los iguales y los de la tabla de la izquierda lo muestra con el campo de la union derecha en null
- right outer join on: muestra todas los iguales y los de la tabla de la derecha lo muestra con el campo de la union izquierda en null

coalesce -> permite asignarle un valor a las columna que tiene "null".
coalesce(columna, 'valor a asignar cuando sea null').

Count se utiliza para contar la cantidad de apariciones que no sean nulas en la tabla segun la condicion.

select count(*) from nomTable where condicion;

count(*) -> cuenta filas.

count(nomColumna) -> cuenta los valores que aparecen en la columna.

sum -> suma

min -> retorna el valor minimo

max -> retorna el valor maximo

avg -> retorna la media

group by -> agrupar valores de una columna. (agrupa segun la primera aparición de los valores)

- Se puede agrupar por más de una columna, pero se debe tener en cuenta el contexto.

- El group by siempre va despues del where.

having -> se utiliza como where en la funciones de grupos.

Subconsultas

Las subconsultas van dentro de parentesis y van en el where (TAMBIEN EN EL FROM).

Verificar que la consulta necesite de una subconsulta simple o multiple.

Subconsultas simples

Las subconsultas simples solo retornan un valor.

Subconsultas multiples

Las subconsultas multiples retornan más de un valor.

Para realizar las subconsultas multiples necesitamos del operador IN, NOT IN, ANY, ALL para poder hacer la comparación con cada valor de retorno de la subconsulta.

Se puede usar MIN y MAX en sustitución de ANY y ALL.

IN	==	= ANY
NOT IN	==	<> ALL
> ANY	==	> select min()
> ALL	==	> select max()

ORACLE SQL

En oracle l'entorn de treball s'anomena PL/SQL

- Para limpiar la terminal: clear screen.
- Mostrar todas las tablas del usuario actual: SELECT table_name FROM user_tables;
- Mostrar todas las tablas del usuario actual y otros usuarios: SELECT table_name FROM all_tables;
- Mostrar todas las tablas en la base de datos (requiere privilegios de DBA): SELECT owner, table_name FROM dba_tables;
- Estructura anomenada BLOC:
 - Cada bloc té 3 parts:
 - declaració: declarem totes les variables, constants, cursors, etc.
 - cos: instruccions / codi.
 - excepcions: tractament d'errors.

- La zona de declaració comença: declare.
- La zona del cos comença: begin.
- La zona excepcions comença: exception.
- Finalizació: end;
- La zona de declaració i excepcions és opcional.
- La zona del cos és obligatoria.

* Mostrar per pantalla (sortida estandard)
DBMS_OUTPUT.PUT_LINE(...)

* Activar la sortida estandard
set serveroutput on

- En el terminal per finalitzar una instrucció utilitzem /
- * Per activar la sortida per pantalla en la terminal: set serveroutput on;
L'activació no és permanent, en cada sessió hauria de fer-la.
- L'assignació es fa amb :=.
- Es pot declara i inicialitzar a la vegada.
- Si la variable no està inicialitzada el seu valor serà null.
- operador de concatenació ||.

* Procedure (procediment)

• Para crear procedimiento se utiliza "create procedure 'nombreDescriptivo' is"
antes de las intrucciones (zonas) y el declare se elimina.

• En terminal con la palabra clave "execute", podemos ejecutar el código guardado en el procedure.

• Para sobrescribir un procedure utilizamos la palabra clave "create or replace

procedure".

- Los procedimientos pueden tener parámetros y su firma es: create procedure nombreDescriptivo(parametro1, parametro2) is.

- Los procedimientos no retornan nada, solo muestran por pantalla.

- * Function (funciones)

- Para crear funciones se utiliza "create function 'nombreDescriptivo' return 'tipoDato' is" antes de las instrucciones (zonas) y el declare se elimina.

- Las funciones no muestran por pantalla, solo retornan un resultado. -- solo aqui.

- La tabla dual permiten probar funciones.

- * Para obtener el tipo de dato sin importa cual sea, por ejemplo cuando cambia de la base de datos, usamos: nomVariable nomTabla.nomColumna%type;

- * Agrupación de datos en plsql se declara de la siguiente manera: nomVariable nomTabla%rowtype y para seleccionar una columna de la agrupación: nomVariable.nomColumna

Esto nos sirve para no tener que declarar una variable por cada campo que necesitemos seleccionar de la tabla y en vez de poner en el select y en el into el nombre de cada columna/variable separado por coma, ponemos el nombre de la variable que tiene la agrupación de los datos.

- Si solo necesitamos seleccionar datos específicos es mejor usar el seleccionado individual, si son todos los datos usamos la agrupación de datos.

Excepciones

Las excepciones descadenan la responsabilidad cuando donde ocurre no se puede gestionar, desde una función o procedimiento si en el mismo no se puede gestionar se le pasa la responsabilidad a oracle, si este no puede le pasa la responsabilidad al sistema operativo, y así sucesivamente hasta que se puede gestionar.

Algunas excepciones:

- zero_divide: gestiona la división por cero como su nombre indica.
- no_data_found: gestiona que el select into no retorna nada
- too_many_rows: gestiona que el select into retorna mas de 1 fila
- dup_val_on_index:

Exception es una zona prohibida de errores que se utiliza para indicar que es el bloque donde se gestionan las excepciones.

Se pueden reversar las operaciones, en la zona de excepciones.

Cargar archivos

@pegar el archivo

commit: para guardar los cambios.

Cursores

* Les consultes que hem fet fins ara només poden retornar 1 resultat.

* Per tractar consultes més complexes tenim els cursors. Per gestionar-los cal:

Declarar

Obrir

Tractar

Tancar

- Per declarar:

CURSOR nom IS consultaSQL;

- Per obrir

OPEN nom;

- Per tractar: Cal fer un bucle

Els cursors ens permeten treballar amb una sèrie de propietats i mètodes:

FETCH: Retorna 1 resultat de la consultat. Va de resultat en resultat.

Quan es fa el fetch, a més, s'activen algunes variables:

NOTFOUND: Booleà que es posa a true (cert) quan el fetch no retorna res

FOUND

NUMROWS: número de fetchs que s'han realitzat

- Per tancar

CLOSE nom;

- Existen dos tipos de cursores (implicitos y explicitos)

* Implicitos: se crean automaticamente cuando se ejecuta una instrucción DML(INSERT, UPDATE, DELETE) o una select into que devuelve una unica fila.

PL SQL gestiona automaticamente la apertura, recuperación y cierre.

* Explicitos: se definen por el programador para consultas que devuelven varias filas.

Se manejan manualmente con DECLARE(CURSOR), OPEN, FETCH y CLOSE.

Sintaxis:

* Cursor con fetch

DECLARE (si no es una función o procedimiento)

CURSOR 'nomDescriptivo' IS

SELECT 'nomColumna'

FROM 'nomTable'

WHERE 'condicion';

'nomPerColumna' 'nomTabla.nomColumna%TYPE';

BEGIN

OPEN 'nomCursor';

LOOP

FETCH 'nomCursor' into 'nomPerColumna';

EXIT WHEN 'nomCursor%NOTFOUND';

DBMS_OUTPUT.PUT_LINE('nomPerColumna');

END LOOP;

CLOSE 'nomCursor';

END;


```
/
* Cursor con bucle for
DECLARE (si no es una función o procedimiento)
    CURSOR 'nomDescriptivo' IS
        SELECT *
        FROM 'nomTable'
        WHERE 'condicion';
BEGIN
    FOR x in 'nomCursor' LOOP
        DBMS_OUTPUT.PUT_LINE(x.nomColumna);
    END LOOP;
END;
/
```

Examen

- -----

- 1 - Procedure / function
 - 2 - Acceso a dades (CRUD)
 - 3 - Excepciones
 - 4 - Cursores